

Following work across a reused buffer

Loop/branch research note

September 18, 2026

1 Three obligations with different answers

A successful synchronization check can pair the wrong work. In the constructed protocol below, replacing all producer input blocks with block zero preserves the barrier matching and native race-free verdict. Consumer occurrences one and two then receive the wrong input. Matching identifies which production supplies a consumer; a separate work contract decides whether that production is intended.

The analysis therefore keeps three questions separate. Does each participant process the intended work item? Does each wait retain its generation target under the schedules admitted by the verifier? Does that wait cover every old reader of the buffer that the producer will next overwrite?

The input contains actual PTX loops and branches. Wasp specializes their fixed input, lowers all 64 threads, checks synchronization, and performs its native memory analysis. This note studies a constructed single-CTA protocol; it does not establish a numerically correct GEMM or the safety of an upstream kernel.

2 From source patterns to a common analysis

The earlier loop/branch record contains 28 concrete cases. They expose a shared problem: participants advance independently, while work identities, barrier phases and physical buffers advance according to different rules. A loop boundary can restart the local work index without restarting the buffer's lifetime. A branch can remove a memory operation while preserving the same barrier arrivals. The analysis must preserve these distinctions before using synchronization to justify reuse.

An *occurrence* identifies one execution of a source instruction by a participant: source site, participant, and dynamic ordinal. Its *work identity* names the intended task, such as $(output, k)$; its evaluated operands name the actual input, buffer address, barrier address and phase parity. These are separate facts. Parity repeats, and a buffer hosts many work items. A *generation* identifies a particular use of a barrier, beyond its parity. A proposed wait-to-generation association becomes *certified matching* only after the verifier checks its stability under the admitted schedules. Finally, *completion coverage* identifies which resource accesses have finished when a particular completion is observed.

This organization follows the source record's existing obligations. Cases A1–A5 and C1–C4 determine occurrences and operands; A2–A3 and B4 ask whether participants agree on work; B3, B6 and D7 examine synchronization handoffs; B1–B2, B5 and E2 distinguish resource-specific completion. D1–D4 track later overwriters, while D5–D6 cover initialization and exit. These case identifiers refer to `loop-branch-discussion.md`, which retains the complete excerpts. They motivate the abstraction; this running implementation does not handle all 28 cases automatically.

The common object is a producer/consumer pipeline whose participants progress independently and reuse asynchronously accessed resources. Its applicability requires sound occurrence and footprint extraction, a stated work contract, stable matching and appropriate completion semantics. Dynamic task allocation needs an additional agreement argument. Schedule-dependent control flow needs a justified

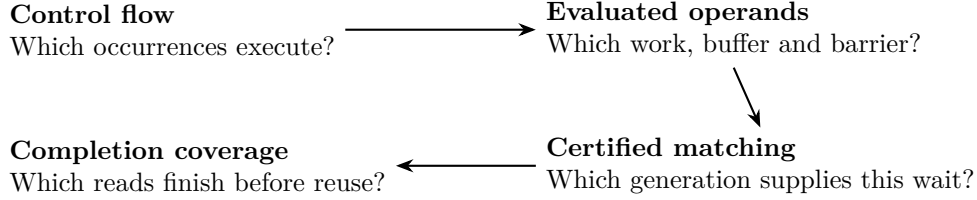


Figure 1: Analysis dependencies. Arrows indicate what must be established before the next claim; they are not event HB edges. Work correspondence is checked against an explicit task contract alongside the evaluated operands.

representation of its alternatives. The current fixed-input example establishes neither a universal fixed DAG nor a smallest loop summary.

3 A small protocol with an output boundary

Two elected threads, in separate warps, issue the producer and consumer operations. All 64 threads participate in initialization and final drain; all 32 consumer-warp lanes collectively allocate and deallocate 128 TMEM columns. MMA disables accumulator input. The async engine has completion steps independent of issuing-thread progress. Two outputs contain two and one work items respectively. Physical shared-memory buffers have 32 KiB capacity each; descriptor-decoded MMA operand footprints must fit inside the selected buffer. The first draft’s 8 KiB capacity failed this check and produced an unintended cross-buffer conflict; it was discarded before the controls below.

Let q be the persistent work ordinal and k the local loop index. For the legal input, $q = 0, 1, 2$ while $(output, k) = (0, 0), (0, 1), (1, 0)$. The buffer index is $q \bmod 2$, and its phase parity is $\lfloor q/2 \rfloor \bmod 2$. The completion barrier for the consumer advances once per MMA and uses $q \bmod 2$. These counters have different meanings even when their early values happen to agree.

The choice $[2, 1]$ makes three distinct boundaries visible with only three work items. Items zero and one use different buffers; item two is both the first work of a new output and the first reuse of buffer zero. Thus local k restarts at precisely the point where the buffer history must continue. Two work items would never test reuse. A single output would never test the restart. The tail then tests that stopping future work retains completion of current work. For item two, the source occurrence binds to work $(1, 0)$, buffer zero and ready-generation one; these associations are recovered from execution operands and certified matching, rather than supplied as a correct-match annotation.

```

producer, for each work q:
  wait empty[buffer(q)] using the previous-use parity
  arrive.expect_tx(full[buffer(q)], 32768)
  issue bulk load(input[q], buffer(q), full[buffer(q)])

consumer, for each work q:
  wait full[buffer(q)] using this-use parity
  issue MMA reading buffer(q)
  commit MMA completion to done
  wait done using q parity
  arrive empty[buffer(q)]
  record completion at output[q]

```

The initial empty wait uses parity one against an initialized phase-zero barrier. Subsequent empty waits observe prior release. The final iteration retains its MMA completion wait and release; loop exit issues no additional copy. There is no numerical accumulator-output claim here. Every final MMA completion precedes the CTA drain and collective TMEM deallocation; the native artifact analyzer discharges the 128 generated lifecycle obligations. The producer has an explicit has-next branch and

stops issuing independently of consumer completion.

4 What the controls distinguish

Change	Native result	Additional obligation
None	Race free	Explicit finite input
Repeat output target zero	Race free	Output coverage fails
Repeat input work zero	Race free	Work correspondence fails
Reset pipeline state	Synchronization candidate	Reuse handoff fails
Release before completion wait	Race candidate	Old read/new write overlap

The reset control changes the pipeline counter at output entry. It preserves the persistent input ordinal and the MMA completion counter, isolating the pipeline-state error. The native checker reports a next-phase arrival before a successful prior-phase wait. This is a synchronization rejection candidate; the complete native execution prefix has been exported and replayed, reproducing the same handoff violation. This establishes a model-level rejection, with no hardware execution claim.

The early-release control moves only the empty arrival across the completion wait. An independent finite protocol projection permits the next producer copy to finish while an old MMA read is outstanding. The native conservative MMA model reports a corresponding shared-memory read/write conflict. Orienting that unordered pair as new-write before old-read gives a full native simulator replay: every requested event executes and the requested order is preserved. This is a witness in the conservative native semantics. The coarse enumeration remains a separate calibration, without a claimed equivalence proof to all PTX schedules.

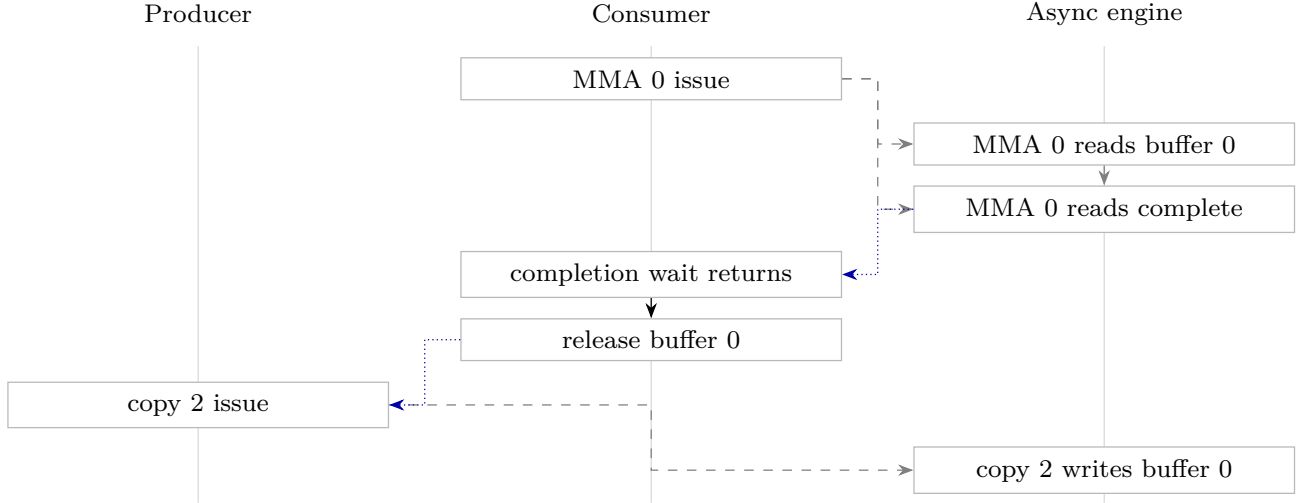


Figure 2: Selected events from the legal native reference execution. Rows give execution order. Solid black denotes program order; dashed gray denotes a completion constraint. Omitted intermediate events remain in the checked graph; dotted blue arrows denote verified HB paths through those events. The old read completes before buffer reuse.

For legal reuse of buffer s , the obligation is

$$\forall r \in \text{Readers}(s, q), \quad r \text{ HB } \text{write}(s, q + 2).$$

Wait matching supplies a synchronization relation. Completion coverage connects that relation to the particular readers. Neither relation supplies work-item identity or output coverage automatically.

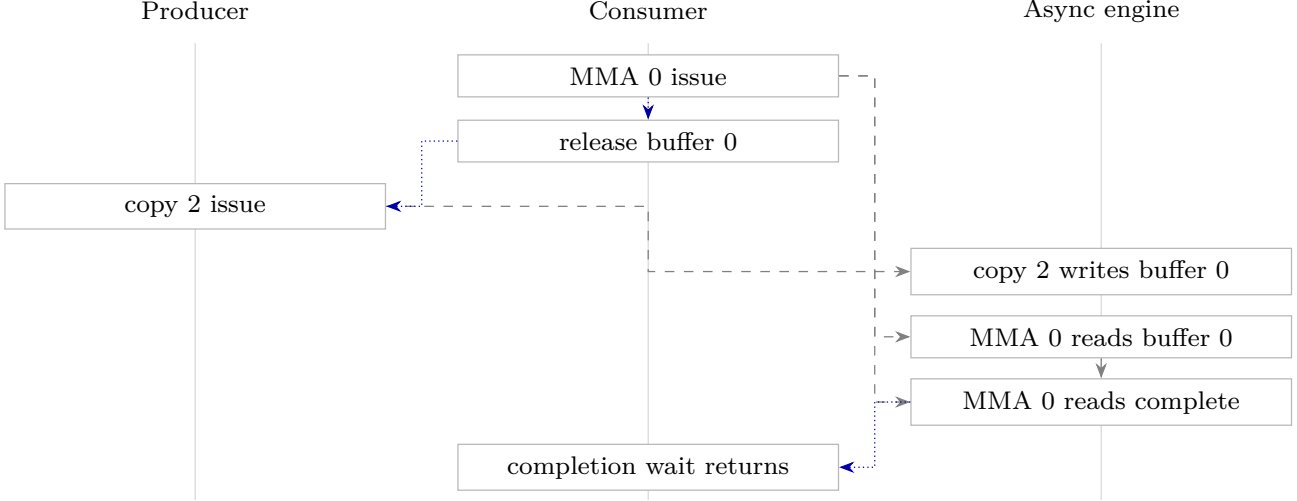


Figure 3: Selected events from the replayed early-release witness. Copy 2 writes buffer 0 before an outstanding MMA 0 read. The frame is generated from the same native event IDs and replay used by the interactive workflow. Vertical order does not itself denote HB.

The binding check follows copy operands and their certified completion phases, then the consumer’s ready wait and every decoded shared-memory read. Its three targets are buffer 0/generation 0, buffer 1/generation 0, and buffer 0/generation 1. Each MMA has 226 modeled shared-read effects. Repeating producer input zero leaves these synchronization targets and output-record locations unchanged, but fails the required input correspondence for consumer occurrences 1 and 2. Moving release early preserves work correspondence while removing old-read coverage at the first reuse. These checks separate the two failure mechanisms on the same event representation.

This conclusion uses four explicit premises: fixed-input control flow yields the modeled occurrences; native checking certifies their generation matching; each ready phase has one recognized producer copy and no unmodeled buffer writer; the decoded read footprint covers the resource use being claimed. For the legal instance, the certificate connects every old read through MMA completion, its committed barrier completion, the consumer wait and release, the producer’s empty wait, and the next write. Moving release before the wait breaks that chain while leaving both waits’ targets intact. Resetting pipeline state instead changes the handoff checked before that chain can be trusted. The first-use parity convention supplies initial credit; final reads are checked against deallocation separately, even when there is no subsequent overwrite.

5 Where the abstraction meets real code

The DeepGEMM source declares pipeline stage and phase outside persistent block scheduling. Its `advance_pipeline` updates the stage and flips parity at ring wrap. The local K index is advanced alongside this state; starting another output does not imply a new physical pipeline. The construction retains this separation, while using smaller work counts. ¹

¹DeepGEMM revision 559d79fb6994a58b8a15b4b93bf13ccc16edf247, [original source](#), lines 176–196 and 270–315.

Listing 1: DeepGEMM: persistent state and local K advancement (lines 183–191).

```
uint32_t stage_idx = 0, phase = 0, tensor_core_phase = 0;
auto advance_pipeline = [&](uint32_t& k_block_idx) {
    ++ k_block_idx;
    // Flip phases only if reach the next first stage
    stage_idx = (stage_idx + 1) % kNumStages;
    phase ^= stage_idx == 0;
};
```

The producer’s outer `while (scheduler.get_next_block(...))` contains a local `k_block_idx = 0` loop. Its increment expression calls `advance_pipeline`. The first action waits on the selected empty barrier with inverted phase parity (lines 197–205). Resetting the local K index consequently leaves the physical pipeline state intact.

FA3’s `fwd_step` supplies an important boundary on generalization. It retains an old V read state, advances the K state, issues QK and PV work, releases K after `warpgroup_wait<1>`, and, on the no-Qv path, releases V after `warpgroup_wait<0>`. One universal “compute done” event would obscure this distinction. Our constructed example deliberately has one completion boundary for its reused operand buffer; FA3 requires resource-specific coverage across distinct chains. ²

Listing 2: FA3: distinct release boundaries. The two excerpts retain their original guards; intervening softmax work is omitted.

```
warpgroup_wait<1>();
pipeline_k.consumer_release(smem_pipe_read); // release K
if constexpr (HasQv) {
    warpgroup_wait<0>();
    pipeline_v.consumer_release(smem_pipe_read_v); // release V
    consumer_wait(pipeline_v, smem_pipe_read);
    // QV GEMM follows on this path.
}
// ... score transformation and softmax ...
if constexpr (!HasQv) {
    warpgroup_wait<0>();
    pipeline_v.consumer_release(smem_pipe_read_v); // release V
}
```

6 HB efficiency: reuse a query result, retain every access

The earlier efficiency record investigated several different costs. *Exact-epoch reasoning* restricts a barrier’s local contributor domain to its justified generation, retaining contributor identities. *Rank-based SMT* represents alternative schedules and a potentially bad prefix with integer positions; its HB masks share symbolic reachability expressions. Certified generation partitions reduced formula construction in the historical run, but both reported solver outcomes remained **unknown**. *Targeted reachability* answers selected concrete HB questions without materializing every pair. Its historical 200 reuse queries cover a narrower observation set than a full memory check, so their timing is not a full-check speedup. These directions and their negative results remain in `hb-efficiency-research-record.md`.

The present experiment continues the concrete-query direction inside Wasp: retain the full memory-check sequence and avoid recomputing identical ancestor information. It does not transfer the historical SMT encoding or claim its unresolved queries are now solved.

The research probe consumes the native synchronization certificate. The default Wasp ancestor-cache policy is compared with an opt-in reuse of the strict-ancestor row for consecutive memory events having identical predecessor lists. Such events have identical strict ancestors in a DAG. Their own accesses have not yet entered the iterator’s previously-seen set, so removing their reflexive bits preserves the emitted access pairs. Every pair is compared in lockstep, including order and identity. This avoids

²FlashAttention revision 77aacb68d194ba9af1010eda5eac3e7c0df8e6f6, [original source](#), lines 1170–1208.

repeated queries without changing the graph or the checked observation set. The option is disabled by default.

For direct predecessors $Pred(v)$ and strict ancestors $Anc(v)$ in a DAG,

$$Anc(v) = \bigcup_{p \in Pred(v)} (\{p\} \cup Anc(p)), \quad Pred(u) = Pred(v) \implies Anc(u) = Anc(v).$$

The iterator still tests each effect against the appropriate previously seen accesses. When v is visited,

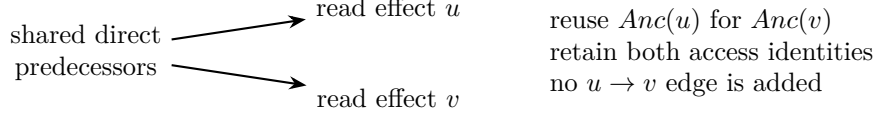


Figure 4: The reuse condition. Equal predecessor lists imply equal strict ancestor sets. Sibling effects remain unordered with respect to each other; sharing their query result does not merge their memory accesses.

u remains a distinct previously seen access; it is not accidentally marked an ancestor of v . Comparing the complete emitted pair sequences checks this operational consequence. The small running example illustrates the condition but uses the eager cache; its reuse-hit count is zero. The larger lazy-cache inputs below measure the actual benefit.

The allocated, fenced, tail-preserving instance has 1,896 events, 6,500 direct edges and 136,326 candidate access pairs. Five measurements follow one warmup, with a 120-second and 8 GiB cap per subprocess. Both single-output and split-output families complete lengths 1, 2, 3, 8, 16, 32 and 64. At 112, the full pair-equality precheck exceeds 120 seconds in both families; that point has no accepted timing comparison. Historical unallocated-instance measurements are kept separately.

Output lengths	Events	Default query (s)	Reuse query (s)
[32]	12,974	0.0080	0.0080
[16, 16]	12,974	0.0081	0.0080
[64]	25,198	28.194	3.278
[32, 32]	25,198	28.672	3.401

The change at 64 coincides with the existing eager ancestor table exceeding its 64 MiB budget and switching to lazy traversal. Reuse then avoids repeated strict ancestor searches for sibling memory effects: 14,528 hits, while retaining all 2,918,231 candidate pairs and 76,222 graph edges. At 32 the eager table fits and the option has zero hits. Output partition changes source control flow while the extracted dependency structure and check count remain the same here.

For [64], query ranges are 27.155–28.748 s/default and 3.120–3.516 s/reuse; memory-check medians are 15.197 s and 3.637 s. These are separate component experiments, without an end-to-end speedup claim. Frontend median is 0.942 s (0.938–0.943); native matching median is 0.119 s. Maximum measured native RSS is 209.3 MiB. The extra strict row requires $8 \lceil \text{accesses}/64 \rceil$ bytes, beyond the old cache counter; process RSS includes it. Fixed-role, legal-tail and work-mismatch inputs also preserve each complete check set; they are distinct specialized inputs, not separate schedules. No SMT solver is called on this concrete-address measurement path.

A migrated PhaseGuard completed-state reachability operator agrees with graph traversal on all 3,594,816 event pairs of each certified small control. Both consume the same native-certified edges. This checks the transferred reachability logic only; frontend extraction, generation checking, and async semantics are not certified equivalent between the two systems by this comparison.

A second comparison executes both verifiers on a common subset: ordinary shared loads/stores, two buffers, three work items, and explicit initial empty credits. Both adapters consume the same extracted source program. Occurrences, physical footprints, wait generations and contributors agree; the early-release conflict is the same third producer write against the first consumer read, and both witnesses replay. PhaseGuard exhausts 181 states for the legal case and 195 for early release. Its adapter contracts the one-time CTA bootstrap to a rendezvous, under checked conditions with no pre-bootstrap memory accesses. This comparison excludes async lowering. In particular, PhaseGuard rejects the main example’s initial opposite-parity shortcut, so that protocol cannot be compared unchanged.

The source-derived task and completion checks above depend on a fixed consumer ordinal contract and native-certified matching. The independent coarse enumerator is not an equivalence proof for the PTX effect graph. Finite successful inputs do not prove arbitrary loop length. A parameterized invariant must retain outstanding readers, both buffer phases, work correspondence and initialization/tail obligations across independently advancing participants.